

CSP-J 初赛风格习题 · 队列 / 栈 / 指针 / 链表

说明：本套题共 20 题，参考 CSP-J（计算机软件能力认证·入门级）初赛题型与难度。

- **第一部分：单项选择题**（第 1-15 题），每题 4 个选项，只有一个正确答案。
- **第二部分：多项选择题**（第 16-20 题），每题 4 个选项，正确答案可能不止一个（漏选得部分分，错选不得分）。
- 链表相关题目只考查**简单的插入、删除操作**，不涉及复杂算法。
- 不含"完善程序"与"程序填空"题型。

第一部分：单项选择题（1-15）

1. 下列关于队列（Queue）的描述中，正确的是（ ）。【队列】

- A. 队列是一种"先进后出"（LIFO）的数据结构
- B. 队列允许在任意位置插入和删除元素
- C. 队列是一种"先进先出"（FIFO）的数据结构，元素从队尾入队、从队首出队
- D. 队列只能存储整数类型的数据

2. 一个初始为空的队列，依次执行：入队 A、入队 B、出队、入队 C、出队。则第二个出队的元素是（ ）。【队列】

- A. A
- B. B

- C. C
- D. 队列已空，不存在第二个出队元素

3. 在使用数组和 `front`、`rear` 指针实现的循环队列中，判断队列为"空"的典型条件是 ()。【队列】

- A. `front == rear`
- B. `front + 1 == rear`
- C. `(rear + 1) % maxSize == front`
- D. `rear == maxSize - 1`

4. 下列应用中，最适合用队列实现的是 ()。【队列】

- A. 程序中的函数递归调用
- B. 表达式求值中的运算符暂存
- C. 广度优先搜索 (BFS) 中待访问结点的暂存
- D. 文本编辑器的"撤销 (Undo)"操作历史

5. 下列关于栈 (Stack) 的描述中，正确的是 ()。【栈】

- A. 栈是一种"先进先出" (FIFO) 的数据结构
- B. 栈只能在栈顶进行插入和删除操作
- C. 栈可以在栈底插入新元素
- D. 栈不能用于函数调用管理

6. 元素依次按 1、2、3 的顺序入栈 (每个元素入栈后可立即出栈，也可稍后出栈)。下列不可能出现的出栈序列是 ()。【栈】

- A. 1 2 3
- B. 3 2 1
- C. 2 3 1

- D. 3 1 2

7. 下列应用中，不适合用栈实现的是（）。【栈】

- A. 程序中的函数调用管理（调用栈）
- B. 表达式中的括号匹配检查
- C. 深度优先搜索（DFS）中回溯路径的保存
- D. 打印机的打印任务排队

8. 依次执行：入栈 P、入栈 Q、出栈、入栈 R、出栈、出栈。则出栈的第二个元素是（）。【栈】

- A. P
- B. Q
- C. R
- D. 不存在第二个出栈元素

9. 在 C++ 中，若有 `int a = 10; int *p = &a;`，则下列说法正确的是（）。【指针】

- A. 变量 `p` 中存储的是整数 10
- B. 变量 `p` 中存储的是变量 `a` 的地址
- C. `*p` 表示指针 `p` 本身的地址
- D. `&a` 表示变量 `a` 的值

10. 对于第 9 题的定义，执行语句 `*p = 20;` 的作用是（）。【指针】

- A. 把指针 `p` 的值改为 20
- B. 把变量 `a` 的值改为 20
- C. 把 `p` 指向的地址改为 20
- D. 编译错误

11. 在 C++ 中，下列关于"空指针"的说法正确的是（ ）。【指针】

- A. 空指针指向地址 0，不能进行任何比较
- B. 空指针可以用 `nullptr`（或 `NULL`）表示，常用于表示指针当前不指向任何有效对象
- C. 空指针和"未初始化的指针"是同一个概念
- D. 对空指针解引用是安全的

12. 若有 `int arr[3] = {5, 6, 7}; int *q = arr;`，则表达式 `*(q + 1)` 的值是（ ）。【指针】

- A. 5
- B. 6
- C. 7
- D. 不确定

13. 下列关于单向链表结点结构的描述，正确的是（ ）。【链表】

- A. 每个结点只包含数据，不包含指向下一个结点的信息
- B. 每个结点通常包含"数据域"和"指针域"两部分，指针域指向下一个结点
- C. 链表的头结点必须存储实际数据
- D. 链表结点之间只能通过数组下标访问

14. 在一个单向链表中，要在表头插入一个新结点 `newnode`（`head` 指向当前首元结点），正确的操作步骤是（ ）。【链表】

- A. 先执行 `newnode->next = head;`，再执行 `head = newnode;`
- B. 先执行 `head = newnode;`，再执行 `newnode->next = head;`
- C. 先执行 `head->next = nullptr;`，再执行 `head = newnode;`

- D. 先释放原 `head`，再令 `head = newnode;`

15. 在单向链表中，要删除结点 `cur`（已知其前驱结点 `prev`，且 `cur` 不是尾结点），正确的操作步骤是（）。【链表】

- A. 执行 `prev->next = cur->next;`，然后释放 `cur` 结点
- B. 直接释放 `cur`，无需修改任何指针
- C. 执行 `cur->next = prev;`，然后释放 `cur`
- D. 执行 `prev->next = nullptr;`，然后释放 `cur`

第二部分：多项选择题（16-20）

16. 下列关于栈和队列的说法中，正确的有（）。【栈 / 队列】

- A. 栈是 LIFO（后进先出）结构，队列是 FIFO（先进先出）结构
- B. 栈和队列都只能用顺序存储（数组）实现，不能用链式存储
- C. 栈和队列都可以用链式存储（链表）实现
- D. 栈常用于表达式求值和括号匹配，队列常用于 BFS 和缓冲区

17. 关于单向链表的优点，下列说法正确的有（）。【链表】

- A. 在已知前驱结点的情况下，在中间插入或删除结点不需要移动大量元素
- B. 可以方便地动态申请和释放内存，链表长度可灵活变化
- C. 随机访问（按下标直接访问）第 i 个元素的时间复杂度是 $O(1)$
- D. 插入、删除操作通常只需要修改若干个指针

18. 在单向链表的表头插入新结点 `newnode`（`head` 指向当前首元结点），下列操作中属于正确步骤的有（）。【链表】

- A. 将 `newnode->next` 指向当前的 `head`

- B. 将 `head` 更新为 `newnode`
- C. 先将原 `head->next` 置为空
- D. 先释放原 `head` 结点

19. 在单向链表中删除某结点 `cur`（已知其前驱结点 `prev`，且 `cur` 不是尾结点），下列操作中属于正确步骤的有（ ）。【链表】

- A. 将 `prev->next` 指向 `cur->next`
- B. 释放（如 `delete`）`cur` 结点所占内存
- C. 将 `cur->next` 指向 `prev`
- D. 不修改任何指针，直接释放 `cur` 即可

20. 下列关于指针安全使用的做法中，正确的有（ ）。【指针】

- A. 指针在使用前应初始化，或明确赋为 `nullptr`
- B. 对动态分配的指针，使用完毕后用 `delete` 释放，并将指针置为 `nullptr`
- C. 直接对未初始化的指针解引用并赋值
- D. 使用指针前先检查其是否为 `nullptr`

参考答案与解析

第一部分：单项选择题

题号	答案	简要解析
1	C	队列 FIFO：队尾入、队首出。A 是栈的特点。
2	B	入 A、入 B → 出 A（第 1 个）；入 C → 出 B（第 2 个）。第二个出队的是 B。

题号	答案	简要解析
3	A	<code>front == rear</code> 表示队空; <code>(rear+1)%maxSize == front</code> 表示队满。
4	C	BFS 用队列按层扩展; 递归/运算符暂存/撤销用栈。
5	B	栈只能在栈顶操作 (LIFO) 。
6	D	若 3 先出, 说明 1、2 已在栈中且未出, 此时栈顶是 2, 必须先出 2 才能出 1, 故 "3 1 2" 不可能。
7	D	打印任务排队是队列 (FIFO) 的典型应用。
8	C	入 P、入 Q → 出 Q (第 1 个) ; 入 R → 出 R (第 2 个) ; 出 P (第 3 个) 。第二个出栈的是 R。
9	B	<code>p</code> 是指针, 存 <code>a</code> 的地址; <code>*p</code> 是 <code>a</code> 的值; <code>&a</code> 是 <code>a</code> 的地址。
10	B	<code>*p</code> 是 <code>p</code> 所指对象 (即 <code>a</code>) , 赋值即修改 <code>a</code> 。
11	B	空指针表示"不指向任何有效对象", 用 <code>nullptr</code> 表示; 对空指针解引用是危险的。
12	B	<code>q</code> 指向 <code>arr[0]</code> , <code>q+1</code> 指向 <code>arr[1]</code> , 值为 6。
13	B	链表结点 = 数据域 + 指针域 (指向下一结点) 。
14	A	必须先让新结点指向原首元 (<code>newnode->next = head</code>) , 再把 <code>head</code> 指向新结点, 顺序不能反。
15	A	让前驱跳过待删结点 (<code>prev->next = cur->next</code>) , 再释放 <code>cur</code> 。

第二部分：多项选择题

题号	答案	简要解析
16	A、C、D	栈与队列都可用顺序或链式存储，B 错误。
17	A、B、D	链表不支持 $O(1)$ 随机访问（C 错误），需顺序遍历。
18	A、B	头插正确步骤：先连后改头。C、D 为错误/多余操作。
19	A、B	删除需先改前驱指针跳过 <code>cur</code> ，再释放 <code>cur</code> 。C、D 错误。
20	A、B、D	未初始化指针解引用（C）是典型错误做法。

题目覆盖：队列（4 题）、栈（5 题）、指针（5 题）、链表（6 题）。链表部分仅考查结点结构、头插法、删除等基础操作。